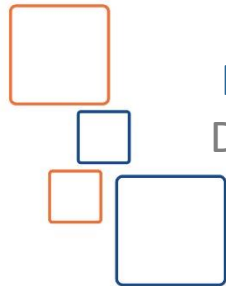


CORE JAVA PROGRAMMING

AN INTRODUCTION TO JAVA



Presented By
Dr. Eman Hesham, DBA.



Java™ Education
and Technology Services



Invest In Yourself,
Develop Your Career

Agenda

- A. The History and Evolution of Java
- B. An Overview of Java
- C. Data Types, Variables and Arrays
- D. Operators, Control Statements and String Handling
- E. Modifiers, Access Specifiers, Packages and Interfaces
- F. Wrapper Classes, Autoboxing and Inner Classes
- G. Exception Handling
- H. Generics and Lambda Expressions and Method Reference
- I. Java Stream API
- J. Java Collections
- K. Multi-Threading in Java

Agenda

- A. The History and Evolution of Java
- B. An Overview of Java
- C. Data Types, Variables and Arrays
- D. Operators, Control Statements and String Handling
- E. Modifiers, Access Specifiers, Packages and Interfaces
- F. Wrapper Classes, Autoboxing and Inner Classes
- G. Exception Handling**
- H. Generics and Lambda Expressions and Method Reference
- I. Java Stream API
- J. Java Collections
- K. Multi-Threading in Java

Lesson 6

Wrapper Classes, Autoboxing and Inner Classes



Inner Classes

Inner Classes

- The Java programming language allows you to define a class within another class.
 - Such a class is called a *nested class* [*Inner Class*].

```
class OuterClass
{
    ...
    class InnerClass
    {
        ...
    }
}
```

Why Use Inner Classes?

- There are several reasons for using inner classes:
 1. It is a way of logically grouping classes that are only used in one place.
 - If a class is useful to only one other class, then it is logical to embed it in that class and keep the two together.
 - Nesting such "helper classes" makes their package more streamlined.

Why Use Inner Classes?

- There are several reasons for using inner classes:

2. It increases encapsulation.

- Consider two top-level classes, A and B, where B needs access to private members of A. By hiding class B within class A, A's members can be declared private and B can access them.
- In addition, B itself can be hidden from the outside world.

```
class A
{
    ...
    class B
    {
        ...
    }
}
```

Why Use Inner Classes?

- There are several reasons for using inner classes:
 3. Nested classes can lead to more readable and maintainable code.
- Nesting small classes within top-level classes places the code closer to where it is used.

Types of Inner Classes

- There are broadly four types of inner classes:
 1. Normal Member Inner Class
 2. Static Member Inner Class
 3. Local Inner Class (inside method body)
 4. Local Anonymous Inner Class

1. Normal Member Inner Class

```
public class OuterClass{  
    private int x ;  
    public void myMethod() {  
        MyInnerClass m = new MyInnerClass() ;  
        m.aMethod() ;  
        ..  
    }  
    public class MyInnerClass{  
        public void aMethod() {  
            //you can access private members of the outer class here  
            x = 3 ;  
        }  
    }  
}
```

1. Normal Member Inner Class

- In order to create an object of the inner class you need to use an object of the outer class.
- The following line of code could have been written inside the method of the enclosing class:

```
MyInnerClass m = this.new MyInnerClass();
```

- The following line of code is used to create an object of the inner class outside of the enclosing class:

```
OuterClass obj = new OuterClass() ;  
OuterClass.MyInnerClass m = obj.new MyInnerClass();
```

1. Normal Member Inner Class

- An inner class can extend any class and/or implement any interface.
- An inner class can assume any accessibility level:
 - private, (friendly), protected, or public.
- An inner class can have an inner class inside it.
- When you compile the java file, two class files will be produced:
 - **MyClass.class**
 - **MyClass\$MyInnerClass.class**
- The inner class has an implicit reference to the outer class.

1. Normal Member Inner Class

The inner class has an implicit reference to the outer class

```
public class MyClass{
    private int x ;
    public void myMethod() {
        MyInnerClass m = new MyInnerClass() ;
        m.aMethod() ;
    }
    class MyInnerClass{
        int x ;
        public void aMethod() {
            x = 10 ;    //x of the inner class
            MyClass.this.x = 25 ;    // x of the outer class
        }
    }
}
```

2. Static Inner Class

- You know, The normal inner class has implicitly a reference to the outer class that created it.
 - If you don't need a connection between them, then you can declare the **inner class static**.
- **A static inner class** means:
 - You don't need an outer-class object in order to create an object of a static inner class.
 - You can't access an outer-class object from an object of a static inner class.

2. Static Inner Class

- Static Inner Class:
 - is among the static members of the outer class.
 - When you create an object of static inner class, you don't need to use an object of the outer class (remember it's static!).
 - Since it is static, such inner class will only be able to access the static members of the outer class.

2. Static Inner Class (Example)

```
public class OuterClass{  
    int x ;  
    static int y;  
    public static class InnerClass{  
        public void aMethod(){  
            y = 10;           // OK  
            x = 33;           // wrong  
        }  
    }  
}
```

```
InnerClass ic= new InnerClass(); // inside outer class  
OuterClass.InnerClass ic= new OuterClass.InnerClass();
```

3. Local Inner Class

```
public class MyClass {  
    private int x ;  
    public void myMethod(final String str, final int a){  
        final int b;  
        class MyLocalInnerClass{  
            public void aMethod(){  
                //you can access private members of the outer class  
                //and you can access final local variables of the method  
            }  
        }  
        MyLocalInnerClass myObj = new MyLocalInnerClass();  
    }  
}
```

3. Local Inner Class

- The object of the local inner class can only be created below the definition of the local inner class (within the same method).
- The local inner class can access the member variables of the outer class.
- It can also access the local variables of the enclosing method if they are declared final.

4. Anonymous Inner Class

```
public class MyClass
{
    public static void main(String []args)
    {
        Timer t = new Timer(10000, new ActionListener() {
            public void actionPerformed(ActionEvent ae) {
                System.out.println("At the tone, the time is " + new
                    Date());
                Toolkit.getDefaultToolkit().beep();
            }
        });
    }
}
```

4. Anonymous Inner Class

- The whole point of using an anonymous inner class is to implement an interface or extend a class and then override one or more methods.
- Of course, it does not make sense to define new methods in anonymous inner class; how will you invoke it?
- When you compile the java file, two class files will be produced:
 - MyClass.class
 - MyClass\$1.class

Lesson 7

Exception Handling



What are Exceptions?

- ▶ Exceptions are **events** that occur during the **execution** of programs that **disrupt** the normal flow of instructions (e.g. divide by zero, array access out of bound, etc.).
- ▶ In Java, an exception is an object that wraps an error event that occurred within a method and contains:
 - ▶ Information about the error including its type
 - ▶ The state of the program when the error occurred
 - ▶ Optionally, other custom information
- ▶ Exception objects can be thrown and caught.
- ▶ Exceptions are used to indicate many different types of error conditions.
- ▶ JVM Errors:
 - ▶ `OutOfMemoryError`, `StackOverflowError`, `LinkageError`
- ▶ System errors:
 - ▶ `FileNotFoundException`, `IOException`, `SocketTimeoutException`
- ▶ Programming errors:
 - ▶ `NullPointerException`, `ArrayIndexOutOfBoundsException`, `ArithmeticException`

Checked vs. Unchecked Exceptions

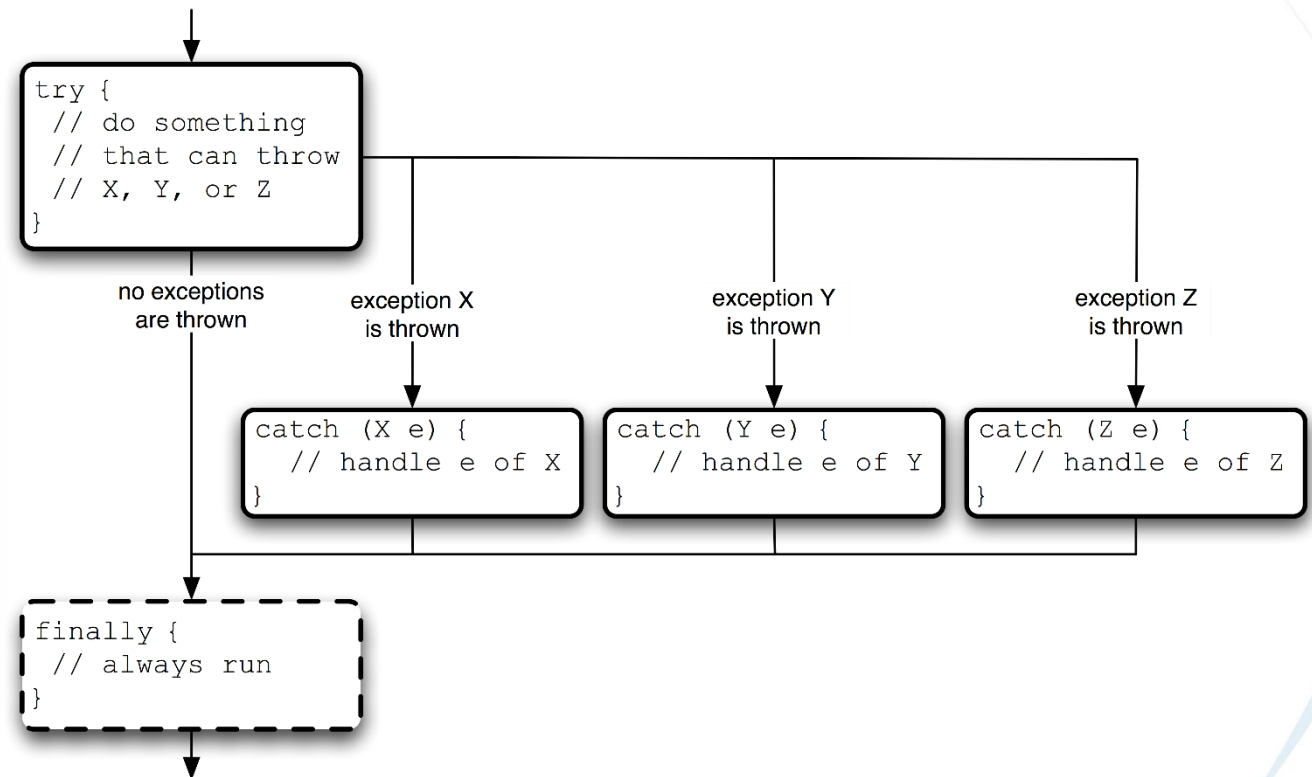
- ▶ **Errors** and **RuntimeExceptions** are **unchecked** – that is, the compiler does not enforce (check) that you handle them explicitly.
- ▶ Methods do not have to declare that they throw them (in the method signatures).
- ▶ It is assumed that the application cannot do anything to recover from these exceptions (at runtime).
- ▶ All other Exceptions are **checked** – that is, the compiler enforces that you handle them explicitly.
- ▶ **Methods** that generate **checked** exceptions must declare that they **throw** them.
- ▶ Methods that invoke other methods that throw checked exceptions should **handle** them (they can be reasonably expected to recover) .

Checked vs. Unchecked Exceptions

- ▶ Checked exceptions give API designers the power to force programmers to deal with the exceptions. API designers expect programmers to be able to reasonably recover from those exceptions, even if that just means logging the exceptions and/or returning error messages to the users.
- ▶ https://www.protechtraining.com/content/java_fundamentals_tutorial-exceptions

Handling Exceptions

```
try {
    // Code block
}
catch (ExceptionType1 e1) {
    // Handle ExceptionType1 exceptions
}
catch (ExceptionType2 e2) {
    // Handle ExceptionType2 exceptions
}
// ...
finally {
    // Code always executed after the
    // try and any catch block
}
```



Handling Exceptions

```
InputStreamReader inputStreamReader = new InputStreamReader(System.in);
BufferedReader reader = new BufferedReader(inputStreamReader);
System.out.println("Type name:");
```

unreported exception IOException; must be caught or declared to be thrown

(Alt-Enter shows hints)

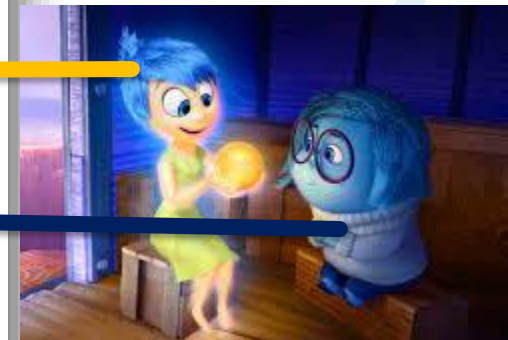
```
name = reader.readLine();
System.out.println("Hello " + name);
```

```
public String readLine() throws IOException {
```

```
String name;
```

```
try {
    name = reader.readLine();
} catch (IOException ex) {
    Logger.getLogger(Lecture_Demo.class.getName()).log(Level.SEVERE, null, ex);
}

System.out.println("Hello " + name);
```



Considerations Regarding Exceptions

- ▶ If several method calls throw different exceptions,
 - ▶ then you can do either of the following:
 1. Write separate **try-catch** blocks for each method.
 2. Put them all inside the same **try** block and then write multiple **catch** blocks for it
(one **catch** for each exception type).
 3. Put them all inside the same **try** block and then just **catch** the parent of all exceptions: **Exception**.

Considerations Regarding Exceptions

- ▶ If more than one `catch` block are written after each other,
 - ▶ then you must take care not to handle a parent exception before a child exception
 - ▶ (i.e. a parent should not mask over a child).
 - ▶ Anyway, the compiler will give an error if you attempt to do so.

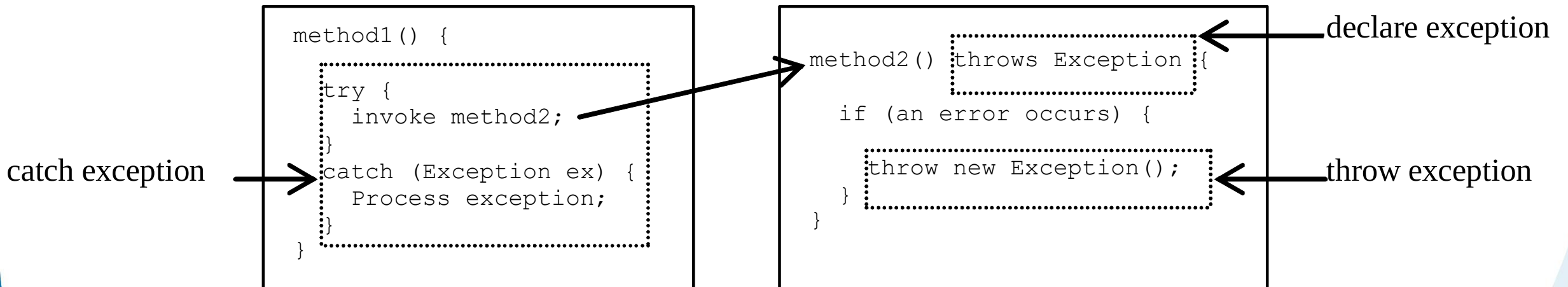
Example

```
try { ..... }  
catch (FileNotFoundException e) {  
    System.err.println("FileNotFoundException: ");  
}  
catch (IOException e) {  
    System.err.println("Caught IOException: ");  
}
```

Or:

```
try { ..... }  
catch (FileNotFoundException | IOException e) {  
    System.err.println(".....");  
}
```

Declaring, Throwing, and Catching Exceptions



Declaring Exceptions

Every method must state the types of checked exceptions it might throw. This is known as *declaring exceptions*.

```
public void myMethod() throws IOException  
{  
}
```

```
public void myMethod() throws IOException, OtherException  
  
{  
  
}
```

Throwing Exceptions

When the program detects an error, the program can create an instance of an appropriate exception type and throw it. This is known as *throwing an exception*. Here is an example,

```
throw new TheException();
```

```
TheException ex = new TheException();  
throw ex;
```

Throwing Exceptions Example

```
/** Set a new radius */  
public void setRadius(double newRadius) throws IllegalArgumentException  
{  
  
    if (newRadius >= 0)  
        radius = newRadius;  
  
    else  
  
        throw new IllegalArgumentException("Radius cannot be negative");  
  
}
```

Catching Exceptions

```
try {  
    obj.setRadius();    // Statements that may throw exceptions  
  
}  
catch (Exception1 exVar1) {  
  
    handler for exception1;  
  
}
```

Catch or Declare Checked Exceptions

Suppose p2 is defined as follows:

```
void p2() throws IOException {  
    if (a file does not exist) {  
        throw new IOException("File does not exist");  
    }  
  
    ...  
}
```

Catch or Declare Checked Exceptions

- Java forces you to deal with checked exceptions.
- If a method declares a checked exception (i.e., an exception other than Error or RuntimeException), you must invoke it in a try-catch block or declare to throw the exception in the calling method.
- For example, suppose that method p1 invokes method p2 and p2 may throw a checked exception (e.g., IOException), you have to write the code as shown in (a) or (b).

```
void p1() {  
    try {  
        p2();  
    }  
    catch (IOException ex) {  
        ...  
    }  
}
```

(a)

```
void p1() throws IOException {  
    p2();  
}
```

(b)

Exception Propagation

```
public class Example3
{
    public static void main(String[] args)
    {
        Example3 ref=new Example3();
        try {
            ref.myMethod1();
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }

    public void myMethod1() throws Exception
    {
        myMethod(5);
    }

    public void myMethod(int x) throws Exception
    {
        if (x>0) throw new Exception("Exception in
                                   method");
    }
}
```

Method Call Propagation

Exception
Propagation

Example: Create A new Exception

```
public class NegativeNumberException extends Exception
{
    public NegativeNumberException()
    {
        super("It is Negative Number could not handle ...");
    }
}
```

Example: Create A new Exception

```
class TestException
{
    public static void main(String []args)
    {
        TestException t = new TestException();
        t.computeFunction();
    }
}
```

Example: Create A new Exception

```
class TestException
{
    public void computeFunction()
    {
        System.out.println("Compute Function is running");
        try{
            System.out.println("Fact (-5) = " + fact(-5));
        }
        catch (NegativeNumberException e)
        {
            e.printStackTrace();
        }
    }
}
```

Example: Create A new Exception

```
int fact(int n) throws NegativeNumberException
{
    if(n<0)    throw new NegativeNumberException();
    if(n==1)   return 1;
    return n*fact(n-1);
}
}
```

Example: Create A new Exception

 D:\Program Files\JCreator Pro\GE2001.exe

```
Compute Function is running
NegativeNumberException: It is Negative Number could not deal ...
    at TestException.fact(NegativeNumberException.java:20)
    at TestException.computeFunction(NegativeNumberException.java:30)
    at TestException.main(NegativeNumberException.java:14)
Press any key to continue...
```

Notes:

- ▶ An exception is considered to be one of the parts of a method's signature. So, when you override a method that throws a certain exception, you have to take the following into consideration:
 - ▶ You may throw the same exception.
 - ▶ You may throw a subclass of the exception
 - ▶ You may decide not to throw an exception at all
 - ▶ You **CAN'T** throw any different exceptions other than the exception(s) declared in the method that you are attempting to override

Agenda

- A. The History and Evolution of Java
- B. An Overview of Java
- C. Data Types, Variables and Arrays
- D. Operators, Control Statements and String Handling
- E. Modifiers, Access Specifiers, Packages and Interfaces
- F. Wrapper Classes, Autoboxing and Inner Classes
- G. Exception Handling
- H. Generics and Lambda Expressions and Method Reference**
- I. Java Stream API
- J. Java Collections
- K. Multi-Threading in Java

Lesson 8

Generics and Lambda Expressions and Method Reference



Generic Programming

Introduction



Why generic Programming

- ***Generic programming*** means writing code that can be reused for objects of many different types.
- Generic classes are desirable because they let you write code that is safer and easier to read than code littered with Object variables and casts.
- Generics are particularly useful for collection classes, such as the universal ArrayList.
- Generic classes are—at least on the surface—similar to templates in C++. In C++, as in Java, templates were first added to the language to support strongly typed collections.

Why generic Programming

- It is important to understand that Java has always given you the ability to create generalized classes, interfaces, and methods by operating through references of type **Object**.
- Because **Object** is the superclass of all other classes, an **Object** reference can refer to any type object.
- In pre-generics code, generalized classes, interfaces, and methods used **Object** references to operate on various types of objects.
- The problem was that they could not do so with type safety.

Why generic Programming

- In other words, the term generics means parametrized types.
- *Parameterized types are important because they enable you to create classes, interfaces, and methods in which the type of data upon which they operate is specified as a parameter.*
- Using generics, it is possible to create a single class, for example, that automatically works with different types of data.
- A class, interface, or method that operates on a parameterized type is called *generic*, as in *generic class* or *generic method*.

The Advantages of Type Parameters

- Before generic classes were added to Java, generic programming was achieved with *inheritance*.
- The ArrayList class simply maintained an array of Object references:

```
public class ArrayList // before generic classes
{
    private Object[] elementData;
    . . .
    public Object get(int i) { . . . }
    public void add(Object o) { . . . }
}
```

The Advantages of Type Parameters

- This approach has two problems. A cast is necessary whenever you retrieve a value:

```
ArrayList files = new ArrayList();
```

```
. . .
```

```
String filename = (String) files.get(0);
```

- Moreover, there is no error checking. You can add values of any class:

```
files.add(new File(". . ."));
```

The Advantages of Type Parameters

- The call in the previous slide compiles and runs without error.
- Elsewhere, casting the result of **get** to a String will cause an error.
- Generics offer a better solution: **type parameters**. The **ArrayList** class now has a type parameter that indicates the element type:
 - `ArrayList<String> files = new ArrayList<String>();`
- This makes your code easier to read. You can tell right away that this particular array list contains String objects.

The Advantages of Type Parameters

- The compiler can make good use of the type information too. No cast is required for calling ***get***. The compiler knows that the return type is `String`, not `Object`:
 - ***String filename = files.get(0);***
- The compiler also knows that the ***add*** method of an ***ArrayList<String>*** has a parameter of type ***String***.
- That is a lot safer than having an `Object` parameter. Now the compiler can check that you don't insert objects of the wrong type. For example, the statement
- ***files.add(new File(".")); // can only add String objects to an ArrayList<String>***
- will not compile. A compiler error is much better than a class cast exception at runtime.



Lab Exercises

Lab Exercise

- Develop an application to convert temperature from Celsius to Fahrenheit using Function<T,R> using inner classes

```
public class TempMain {  
    public static void main(String[] args) {  
        float x=24;  
  
        float out= new Function<Float, Float>() {  
            @Override  
            public Float apply(Float t) {  
                return                 ;  
            }  
  
        }.apply(t: x);  
    }  
}
```

Lab Exercise

- Create your own exception class.
- Write down two other classes:
 - the first will contain three methods throwing your newly created exception class
 - the second class will be calling the methods that throws exception using the try-catch-finally block.

Lab Exercise

- Create a base class named Shape that contains one abstract method draw().
- Create two concrete classes (Rectangle and Circle) that extend Shape
- Create a test class that defines a method that accepts a list that contains **only** child classes of shape
- Test your method by creating two ArrayList of Rectangle and shapes and pass them to the generic method

Lab Exercise

- Create a generic interface that could be used to represent complex numbers
- Create some generic methods that represent basic arithmetic operation on complex.

```
public interface Complex<T> {  
  
    T getReal();  
    T getImaginary();  
  
    Complex<T> add(Complex<T> z);  
    Complex<T> subtract(Complex<T> z);  
    Complex<T> product(Complex<T> z);  
    Complex<T> div(Complex<T> z);  
  
}
```

```
public class DoubleComplex implements Complex<Double> {
```